SY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
PRACTICAL ASSIGNMENT TEMPLATE

1. Student Details

Student Name	Pratik Akrur Pandit
PRN	202501110116
Division	B
Batch	B2
Roll No.	202501110116
Date of Submission	27/10/2026

2. Practical Details

Practical No.	11
Practical Title	Standard Template Library (list and vector containers)
Date Performed	24/10/2026
Faculty Name	Shruti Gadewar

3. Problem Statement

Write C++ programs based on the Standard Template Library (STL): (a) an assignment using the list class as a container of the STL, and (b) an assignment using the vector class as a container of the STL. The programs must perform common operations such as insertion, deletion, searching, sorting and traversal using member functions, iterators and STL algorithms.

4. Objectives

1. To understand the four components of the STL: algorithms, containers, functions (functors) and iterators.
2. To use the list class (a sequence container) and perform operations on it.
3. To use the vector class (a sequence container) together with iterators and the <algorithm> functions sort(), reverse() and find().

5. Expected Learning Outcomes

After completing this practical, the student will be able to:

1. Explain what the STL is and list its components and types of containers.
2. Create, modify and traverse list and vector containers using member functions and iterators.
3. Differentiate between list and vector and choose the suitable container for a given problem.

6. Theory / Concept

Explain the important concepts, algorithms, techniques, or methods used.

The **Standard Template Library (STL)** is a set of C++ template classes that provide common programming data structures and functions such as lists, stacks and arrays. It is a library of container classes, algorithms and iterators. It is a generalized library, so its components are parameterized; a working knowledge of template classes is a prerequisite for working with the STL.

STL has four components:

- **Algorithms:** the <algorithm> header defines a collection of functions designed to be used on ranges of elements (for example sorting and searching). They act on containers and provide various operations on their contents.
- **Containers:** container classes store objects and data. There are seven standard first-class container classes and three container adaptor classes.
- **Functions:** the STL includes classes that overload the function call operator. Instances of such classes are called function objects or functors; they allow the working of the associated function to be customized through parameters.
- **Iterators:** iterators are used for working upon a sequence of values. They are the major feature that allows generality in the STL.

Types of containers:

- **Sequence containers** (accessed sequentially): vector, list, deque, arrays.
- **Container adaptors** (provide a different interface for sequential containers): queue, priority_queue, stack.
- **Associative containers** (sorted data structures that can be quickly searched, O(log n)): set, multiset, map, multimap.
- **Unordered associative containers** (unordered data structures that can be quickly searched): unordered_set, unordered_multiset, unordered_map, unordered_multimap.

list is a sequence container implemented as a doubly linked list. Elements are not stored contiguously, so there is no random access (no [ ] operator), but insertion and deletion at any known position is fast. Useful member functions include push_back(), push_front(), pop_back(), pop_front(), insert(), remove(), sort(), reverse(), front(), back() and size().

vector is a sequence container implemented as a dynamic array. Elements are stored contiguously, so any element can be accessed directly using [ ] or at(), and elements can be added at the end quickly using push_back(). Insertion or deletion in the middle is slower because the following elements have to be shifted. Useful member functions include push_back(), pop_back(), insert(), erase(), at(), size(), empty() and clear().

7. Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode for the proposed solution.

START

Step 1: Include <iostream>, <list>, <vector> and <algorithm>; define functions to display a list and a vector using iterators.

Step 2: Part A: create a list of integers; use push_back(), push_front(), size(), front() and back(); then apply sort(), reverse(), insert(), remove(), pop_front() and pop_back(), displaying the list after each operation.

Step 3: Part B: create a vector of integers using push_back(); display its size and elements using [ ] and at().

Step 4: Apply sort(), reverse() and find() from <algorithm> on the vector, and then use insert(), erase() and pop_back().

Step 5: Calculate the sum of the remaining elements, clear the vector and display the final size and whether it is empty.

STOP

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Programming Language	C++
IDE / Platform	Dev-C++ (Windows)
Libraries / Packages	<iostream> (input/output), <list> (list container), <vector> (vector container), <algorithm> (sort, reverse, find)

09. Implementation / GitHub Repository

GitHub Repository URL	https://github.com/panditpratik/OOP-
Repository Name	**Problem-Solving-Using-OOP-in-Cpp**
Branch Used	main
Commit ID / Latest Commit	0ae88150a210e316a425746b2cb833bb87f37e91
Date of Last Commit	27/10/26

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Actual Output	Status
TC1 list push_back / push_front	30, 10, 50 and push_front(20)	20 30 10 50	20 30 10 50	Pass
TC2 list size, front, back	-	Size = 4, Front = 20, Back = 50	Size = 4, Front = 20, Back = 50	Pass
TC3 list sort()	20 30 10 50	10 20 30 50	10 20 30 50	Pass
TC4 list reverse()	10 20 30 50	50 30 20 10	50 30 20 10	Pass
TC5 list insert(40)	at position 2	50 40 30 20 10	50 40 30 20 10	Pass
TC6 list remove(20)	50 40 30 20 10	50 40 30 10	50 40 30 10	Pass
TC7 list pop_front / pop_back	50 40 30 10	40 30 (final size = 2)	40 30 (final size = 2)	Pass
TC8 vector push_back	25, 5, 15, 10	25 5 15 10 (size = 4)	25 5 15 10 (size = 4)	Pass
TC9 vector sort() and reverse()	25 5 15 10	5 10 15 25, then 25 15 10 5	5 10 15 25, then 25 15 10 5	Pass

TC10 vector find()	search for 10	Element 10 found at index 2	Element 10 found at index 2	Pass
TC11 vector insert, erase, pop_back	insert(20) at index 1	20 15 10, sum of elements = 45	20 15 10, sum of elements = 45	Pass
TC12 vector clear()	-	size = 0, empty = Yes	size = 0, empty = Yes	Pass

For ML practicals, include appropriate performance metrics, graphs, tables, or model outputs wherever applicable.

11. Results / Observations

Mention the important results obtained during execution.

- Elements were inserted at the front and back of the list using push_front() and push_back(), and the list member functions sort(), reverse() and remove() changed the list as expected.
- The list does not provide random access, so it was traversed using iterators; insert() placed a new element before the position pointed to by the iterator.
- The vector allowed direct access to elements using [ ] and at(), and the STL algorithms sort(), reverse() and find() worked on the range v.begin() to v.end().
- Both containers were displayed using iterators, which shows that the same traversal approach works for different containers.

Screenshots / Graphs / Output:

Practical_assignment_11.cpp > main()

```cpp
#include <iostream>
#include <list>
#include <vector>
#include <algorithm>
using namespace std;

void displayList(const list<int> &l)
{
    for (list<int>::const_iterator it = l.begin(); it != l.end(); ++it)
        cout << *it << " ";

    cout << endl;
}

void displayVector(const vector<int> &v)
{
    for (vector<int>::const_iterator it = v.begin(); it != v.end(); ++it)
        cout << *it << " ";

    cout << endl;
}

int main()
{
    // ================= PART A : list container =================

    cout << "===== PART A: list class as a container =====" << endl;

    list<int> myList;

    myList.push_back(30);
    myList.push_back(10);
    myList.push_back(50);
    myList.push_front(20);

    cout << "After push_back / push_front : ";
    displayList(myList);

    cout << "Size = " << myList.size() << endl;
    cout << "Front = " << myList.front()
         << ", Back = " << myList.back() << endl;

    myList.sort();
    cout << "After sort(): ";
    displayList(myList);

    myList.reverse();
    cout << "After reverse(): ";
```

```
    cout << "After reverse(): ";
    displayList(myList);

    list<int>::iterator it = myList.begin();
    ++it; // Points to the 2nd element

    myList.insert(it, 40);

    cout << "After insert(40) at position 2: ";
    displayList(myList);

    myList.remove(20);

    cout << "After remove(20): ";
    displayList(myList);

    myList.pop_front();
    myList.pop_back();

    cout << "After pop_front(), pop_back() : ";
    displayList(myList);

    cout << "Final size = " << myList.size() << endl;

    // ================ PART B : vector container ================

    cout << endl << "===== PART B: vector class as a container =====" << endl;

    vector<int> v;

    v.push_back(25);
    v.push_back(5);
    v.push_back(15);
    v.push_back(10);

    cout << "After push_back: ";
    displayVector(v);

    cout << "Size = " << v.size()
         << ", v[1] = " << v[1]
         << ", v.at(2) = " << v.at(2) << endl;

    sort(v.begin(), v.end());

    cout << "After sort(): ";
    displayVector(v);

```

```
    reverse(v.begin(), v.end());

    cout << "After reverse(): ";
    displayVector(v);

    vector<int>::iterator pos = find(v.begin(), v.end(), 10);

    if (pos != v.end())
        cout << "Element 10 found at index "
             << (pos - v.begin()) << endl;

    v.insert(v.begin() + 1, 20);

    cout << "After insert(20) at index 1: ";
    displayVector(v);

    v.erase(v.begin());

    cout << "After erase() of first element: ";
    displayVector(v);

    v.pop_back();

    cout << "After pop_back(): ";
    displayVector(v);

    int sum = 0;

    for (vector<int>::size_type i = 0; i < v.size(); i++)
        sum += v[i];

    cout << "Sum of elements = " << sum << endl;

    v.clear();

    cout << "After clear(): size = " << v.size()
         << ", empty = " << (v.empty() ? "Yes" : "No") << endl;

    return 0;
}
```

- Output:-

```
PS D:\All Programs\OOP(C++)> cd "d:\All Programs\OOP(C++)\" ;
===== PART A: list class as a container =====
After push_back / push_front : 20 30 10 50
Size = 4
Front = 20, Back = 50
After sort(): 10 20 30 50
After reverse(): 50 30 20 10
After insert(40) at position 2: 50 40 30 20 10
After remove(20): 50 40 30 10
After pop_front(), pop_back() : 40 30
Final size = 2

===== PART B: vector class as a container =====
After push_back: 25 5 15 10
Size = 4, v[1] = 5, v.at(2) = 15
After sort(): 5 10 15 25
After reverse(): 25 15 10 5
Element 10 found at index 2
After insert(20) at index 1: 25 20 15 10 5
After erase() of first element: 20 15 10 5
After pop_back(): 20 15 10
Sum of elements = 45
After clear(): size = 0, empty = Yes
PS D:\All Programs\OOP(C++)>
```

12. Conclusion

Write a brief conclusion describing what was implemented and what was learned.

In this practical, programs based on the Standard Template Library were implemented using the list and vector classes as containers. Operations such as insertion, deletion, searching, sorting, reversing and traversal were performed using member functions, iterators and the algorithms of the STL. I learned that the STL provides ready-made, reusable and efficient data structures and algorithms, and that list is better for frequent insertion and deletion while vector is better for fast random access.

13. Assignment Questions

Q1. What is the Standard Template Library? Explain its components.

Answer: The STL is a set of C++ template classes that provide common data structures and functions such as lists, stacks and arrays. It is a generalized library whose components are parameterized. It has four components: (1) **Algorithms**, functions defined in the <algorithm> header that work on ranges of elements, for example sorting and searching; (2) **Containers**, classes that store objects and data, such as vector, list, deque, stack, queue, set and map; (3) **Functions** (functors), objects of classes that overload the function call operator so that the working of a function can be customized with parameters; and (4) **Iterators**, which are used to work upon a sequence of values and give generality to the STL.

Q2. Differentiate between the list and vector containers.

Basis	**list**	**vector**
Implementation	Doubly linked list	Dynamic array
Basis	**list**	**vector**
Memory	Non-contiguous	Contiguous
Element access	No random access; only through iterators	Random access using [ ] and at()
Insertion / deletion	Fast at any known position	Fast at the end; slow in the middle (elements are shifted)
Sorting	Member function list.sort()	sort() from <algorithm>

14. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	**PRN**	**Signature**
Pratik Akrur Pandit	202501110116.	